

Question 1

a)

This program runs in $O(n)$ time, because it touches each node only once, and `calculate_sizes` does constant work per node.

b)

1.b. For strong induction, we want to prove proposition $P(n)$ is true.

- base case: $P(2t+1)$ is true
- inductive step: for some $n > 2t+1$, assume that for every m s.t. $2t+1 \leq m < n$, $P(m)$ is true, and use that to prove $P(n)$ is true.

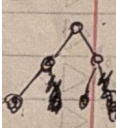
Let $P(n)$ defined as: for binary tree T with $|T| = n$, $\exists$ vertex v s.t. $t \leq |T_v| \leq 2t-1$. $\overset{n \geq 2t+1}{\downarrow}$
subtree at v

Consider the set of vertices for which subtree has size at least t :

$$S = \{v \in T : |T_v| \geq t\}$$

non-empty, since $|T_{\text{root}}| = n \geq 2t+1 \geq t > 0$

at v :
For subtree, $|T_v| = 1 + |T_L| + |T_R|$ for child vert. L & R , if present (0 if not).

 Base case: $n = 2t+1$; $P(2t+1)$

defined above

Let $|T| = 2t+1$. ~~We have~~ Suppose we pick vertex v from S s.t. v is min. subtree. Thus, $|T_v| \geq t$, and for v 's children a & b , $|T_a| < t$ & $|T_b| < t$ (since their subtrees must be smaller than T_v , & since if they were $\geq t$ this would contradict T_v being min.)
 $\therefore |T_a| \leq t-1$, $|T_b| \leq t-1$. $\therefore |T_v| = 1 + |T_a| + |T_b| \leq 1 + (t-1) + (t-1) = 2t-1$
 $\therefore t \leq |T_v| \leq 2t-1$

Inductive step: $n > 2t+1$, $\forall m$ s.t. $2t+1 \leq m < n$, $P(m)$ is true.

Take a tree T with $|T| = n$. We need to find v s.t. $t \leq |T_v| \leq 2t-1$.

Starting at the root, it does not work, since $n > 2t+1$. We can look at the root's children, with sizes ~~x, y~~ x, y . We claim that at least 1 of $x, y \geq t$. If this is not true, then $x \leq t-1$ & $y \leq t-1$ which $\Rightarrow 1 + x + y \leq 1 + (t-1) + (t-1) = 2t-1$; contradicting $n \geq 2t+1$.

$\therefore$ Pick child c with $|T_c| \geq t$. We have 3 cases:

Case 1: $t \leq |T_c| \leq 2t-1$. $v = c$

Case 2: $|T_c| \geq 2t+1$. Then $|T_c| = m$; $2t+1 \leq m < n$ since c is strictly smaller than root. $P(m)$ is true, $\Rightarrow$ there exists a descendant v of c s.t. $t \leq |T_v| \leq 2t-1$

Case 3: $|T_c| \geq 2t$. Consider $S' = \{w \in T_c : |T_w| \geq t\}$
 As reasoned earlier, this set is nonempty, since root c 's $|T_c| = 2t \geq t$.
 We can pick min sub-tree vertex $v \in S'$, thus v 's children inside T_c have subtree size $< t$,
 so $|T_v| \leq 1 + (t-1) + (t-1) = 2t-1$
 AND $|T_v| \geq t$ by def.
 $\therefore P(n)$ holds true for all $n \geq 2t+1$.

c)

My algorithm runs in $O(h)$ time, because starting at vertex v , for each iteration we either go left or right, and there is no backtracking – so in the worst case this would be from the root to the furthest leaf (which is analogous to the height of the tree.) This assumes that every iteration only does $O(1)$ work, with a constant number of pointer checks and reading stored sizes.

Question 2

a) i.

have to show $\exists c > 0$ st. $f(n) \leq c \cdot g(n)$

i) $f(n) = 3(\log_2 n)^{3/2} = 3\left(\frac{\ln n}{\ln 2}\right)^{3/2}$
 $= C(\ln n)^{3/2}$ where $C = \frac{3}{(\ln 2)^{3/2}}$

$g(n) = 2\ln n \times (\ln n + 3)$
 $2(\ln n)^2 + 6\ln n \approx 2(\ln n)^2$

$\frac{f(n)}{g(n)} \approx \frac{C(\ln n)^{3/2}}{2(\ln n)^2} = \frac{C}{2} \cdot \frac{1}{(\ln n)^{1/2}}$ $\lim_{n \rightarrow \infty} \frac{C}{2} \cdot \frac{1}{\sqrt{\ln n}} = 0$

$\therefore f(n) = o(n) \Rightarrow f(n) = O(n)$

ii.

$f(n) = \text{size of } S \subseteq [n] : |S| \leq 4$
 for $[n] = \{0, 1, 2, \dots, n-1\}$

i.e. $f(n)$ number of subsets of size $0 \rightarrow 4$
 in $\#$ in range 0 to $n-1$

$$f(n) = \begin{matrix} \# \text{ subsets of} \\ \text{size } 0 \end{matrix} + \dots + \begin{matrix} \# \text{ subsets of} \\ \text{size } 4 \end{matrix}$$

$$= {}^nC_0 + {}^nC_1 + {}^nC_2 + {}^nC_3 + {}^nC_4$$

size 1:

 ~~nC_1~~

size 2:

 ~~nC_2~~

$$= \frac{n!}{4!(n-4)!}$$

$$\therefore f(n) \geq \binom{n}{4} = \frac{n(n-1)(n-2)(n-3)}{24}$$

suppose $n \geq 6$ Then $(n-1)(n-2)(n-3) \geq n/2$

$$\therefore \binom{n}{4} \geq \frac{n \cdot (n/2) \cdot (n/2) \cdot (n/2)}{24} = \frac{n^4}{192}$$

$$\Rightarrow f(n) \geq \frac{n^4}{192}$$

$$\frac{n^4}{192} > 3cn^3 \iff \frac{n}{192} > 3c \iff n > 576c$$

$cg(n)$

$\therefore$ for any c , if we choose sufficiently large n ,
 $f(n) > cg(n)$, so $f(n) \neq O(n)$
 and $f(n) \neq o(n)$

iii.

iii) $f(n) = n! = \cancel{n(n-1)(n-2)\dots(1)}$ $g(n) = 2^{n^2}$

$$n! \leq n^n$$

$$\leq (2^{\log_2 n})^n = 2^{n \log_2 n}$$

$$\frac{n!}{2^{n^2}} \leq \frac{2^{n \log_2 n}}{2^{n^2}} = 2^{n \log_2 n - n^2}$$

$$\frac{n!}{2^{n^2}} \leq 2^{n \log_2 n - n^2}$$

$\lim_{n \rightarrow \infty} \frac{n!}{2^{n^2}} = 0$ since $-n^2$ approaches $-\infty$, thus $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$

$\therefore f(n) = o(g(n)) \Rightarrow f(n) = O(g(n))$

iv.

iv) Not true. Suppose $f(n) = g(n) = n$.

$f = O(g)$ since $n \leq 2n$

$f \neq o(g)$ since $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 1 \neq 0$

$g \neq o(f)$ same as above.

b) I am apprehensive about writing proofs, and confronting my gaps in knowledge – I feel like I often face a wall when I'm looking at a hard question where the answer isn't immediately obvious, and I also struggle to go through the first steps. A lot of the time, it feels like I'm just missing a bunch of pre-req knowledge that didn't sink in for some reason or other.

I'm excited to confront this head-on, and with the help of the teaching staff! I'm also looking forward to catching up on foundational knowledge that I feel has slipped away over the years.

I'll do my best to communicate transparently when I need help, to ask questions unabashedly and to give a best-effort attempt for every problem! And I'd like to pro-actively note what areas I feel weak in and review them asap so that I can try to stay as on top of things as possible!

c) survey submitted!